

ASSIGNMENT – 13.2

Name: K. Keerthana

H. No:2303A52027

Batch: 38

Task Description -1 (Refactoring – Eliminating Repetitive Logic)

- Task: Use AI-assisted coding techniques to restructure a Python program that performs repeated calculations, transforming it into a reusable and maintainable design.

Instructions:

- Ask AI to locate repeated computations within the program.
- Encapsulate repeated logic into reusable functions.
- Verify that program output remains unchanged after refactoring.
- Add proper docstrings to all defined functions.

Sample Code:

```
length = 8
width = 4
print("Area:", length * width)
print("Perimeter:", 2 * (length + width))

length = 12
width = 6
print("Area:", length * width)
print("Perimeter:", 2 * (length + width))
```

Expected Output:

A refactored Python script that uses a single reusable function to compute area and perimeter without repeating code.

PROMPT:

Refactor the following Python code to remove repeated calculations.

Create a reusable function to calculate area and perimeter.

Make sure the program output remains the same and add a proper docstring to the function.

```
length = 8
width = 4
```

```
print("Area:", length * width)

print("Perimeter:", 2 * (length + width))

length = 12

width = 6

print("Area:", length * width)

print("Perimeter:", 2 * (length + width))
```

CODE:

```
1 def calculate_area_perimeter(length, width):
2     """
3     Calculate the area and perimeter of a rectangle.
4
5     Parameters:
6     length (float): The length of the rectangle.
7     width (float): The width of the rectangle.
8
9     Returns:
10    tuple: A tuple containing the area and perimeter of the rectangle.
11    """
12    area = length * width
13    perimeter = 2 * (length + width)
14    return area, perimeter
15
16 # First rectangle
17 length1 = 8
18 width1 = 4
19 area1, perimeter1 = calculate_area_perimeter(length1, width1)
20 print("Area:", area1)
21 print("Perimeter:", perimeter1)
22 # Second rectangle
23 length2 = 12
24 width2 = 6
25 area2, perimeter2 = calculate_area_perimeter(length2, width2)
26 print("Area:", area2)
27 print("Perimeter:", perimeter2)
```

OUTPUT

```
● Area: 32
  Perimeter: 24
  Area: 72
  Perimeter: 36
```

EXPLANATION:

This program defines a function `calculate_area_perimeter()` to compute the area and perimeter of a rectangle using given length and width. The function returns both values as a tuple. It is then called

for two rectangles (8×4 and 12×6) to avoid repeating the same calculations. This improves code reusability and maintainability.

Task Description -2 (Refactoring – Modularizing Inline Calculations)

- Task: Use AI to refactor a Python script by extracting repeated inline calculations into reusable and well-documented functions.

Instructions:

- Identify repeated calculation patterns in the code.
- Convert them into reusable functions.
- Ensure readability and proper documentation.

Sample Code:

```
amount = 1000
discount = amount * 0.10
final_amount = amount - discount
print("Final Amount:", final_amount)
amount = 2000
discount = amount * 0.10
final_amount = amount - discount
print("Final Amount:", final_amount)
```

Expected Output:

A modular Python program that uses a reusable function to compute final amounts for different inputs.

PROMPT:

Analyze the following Python code and identify repeated calculations.

Refactor the code by creating a reusable function to calculate the final amount after applying a discount.

Ensure the program output remains the same and add a proper docstring to the function for clarity and readability.

```
amount = 1000
discount = amount * 0.10
final_amount = amount - discount
print("Final Amount:", final_amount)
amount = 2000
discount = amount * 0.10
```

```
final_amount = amount - discount  
print("Final Amount:", final_amount)
```

CODE:

```
def calculate_final_amount(amount):  
    """  
    Calculate the final amount after applying a 10% discount.  
  
    Parameters:  
    amount (float): The original amount before discount.  
  
    Returns:  
    float: The final amount after applying the discount.  
    """  
    discount = amount * 0.10  
    final_amount = amount - discount  
    return final_amount  
  
# Calculate and print final amount for the first case  
amount1 = 1000  
final_amount1 = calculate_final_amount(amount1)  
print("Final Amount:", final_amount1)  
# Calculate and print final amount for the second case  
amount2 = 2000  
final_amount2 = calculate_final_amount(amount2)  
print("Final Amount:", final_amount2)
```

OUTPUT:

```
Python 3.10.0 Win32 x64 (bundled)  
● Final Amount: 900.0  
Final Amount: 1800.0
```

EXPLANATION:

This program defines a function `calculate_final_amount(amount)` that calculates the final price after applying a 10% discount. The function computes the discount, subtracts it from the original amount, and returns the final amount. It is then called for two amounts (1000 and 2000) instead of repeating the same calculations. This improves code reusability, readability, and maintainability.

Task Description -3 (Refactoring – Improving Loop and Conditional Performance)

- Task: Apply AI-based suggestions to enhance the performance of

Python code that relies on nested loops and repeated conditional checks.

Instructions:

- Use AI to recommend more efficient data structures or logic simplifications.
- Replace inefficient constructs while preserving program behavior.
- Compare execution efficiency before and after refactoring.

Sample Code:

```
students = ["Anil", "Bhavya", "Charan", "Divya"]
check_list = ["Charan", "Esha", "Bhavya"]
for name in check_list:
    exists = False
    for student in students:
        if name == student:
            exists = True
    if exists:
        print(name, "found")
    else:
        print(name, "not found")
```

Expected Output:

An optimized Python program that performs faster lookups using efficient logic, along with execution performance comparison.

PROMPT:

Refactor the following Python code to improve performance by removing nested loops and repeated condition checks.

Use a more efficient data structure (like a set) while keeping the same output.

Also show a simple comparison of execution efficiency before and after refactoring.

```
students = ["Anil", "Bhavya", "Charan", "Divya"]
check_list = ["Charan", "Esha", "Bhavya"]
for name in check_list:
    exists = False
    for student in students:
        if name == student:
            exists = True
    if exists:
        print(name, "found")
```

else:

print(name, "not found")

CODE:

```
# Refactored Code
students = ["Anil", "Bhavya", "Charan", "Divya"]
check_list = ["Charan", "Esha", "Bhavya"]
# Convert the students list to a set for O(1) average time complexity lookups
student_set = set(students)
for name in check_list:
    if name in student_set:
        print(name, "found")
    else:
        print(name, "not found")
# Performance Comparison
import time
# Original Code Performance
students = ["Anil", "Bhavya", "Charan", "Divya"]
check_list = ["Charan", "Esha", "Bhavya"]
start_time = time.time()
for name in check_list:
    exists = False
    for student in students:
        if name == student:
            exists = True
    if exists:
        print(name, "found")
    else:
        print(name, "not found")
end_time = time.time()
```

```
print("Original code execution time:", end_time - start_time)
# Refactored Code Performance
students = ["Anil", "Bhavya", "Charan", "Divya"]
check_list = ["Charan", "Esha", "Bhavya"]
student_set = set(students)
start_time = time.time()
for name in check_list:
    if name in student_set:
        print(name, "found")
    else:
        print(name, "not found")
end_time = time.time()
print("Refactored code execution time:", end_time - start_time)
```

OUTPUT:

```
Charan found
Esha not found
Bhavya found
Charan found
○ Esha not found
Bhavya found
Original code execution time: 0.0005581378936767578
Charan found
Esha not found
Bhavya found
Refactored code execution time: 0.0017108917236328125
```

EXPLANATION:

This program improves the search performance of student names.

In the original code, a nested loop checks each name in the list, which is slower.

In the refactored code, the students list is converted to a set, so names can be found faster using in.

The time module is used to compare the execution time of the original and optimized code.

Task Description -4 (Refactoring – Replacing Hardcoded Values with Constants)

- Task: Utilize AI assistance to improve maintainability by replacing hardcoded numerical values with named constants.

Instructions:

- Detect hardcoded numeric values used directly in expressions.
- Define descriptive constants at the beginning of the program.
- Update calculations to use these constants without changing results.

Sample Code:

```
print("Simple Interest:", (1000 * 2 * 5) / 100)
```

```
print("Simple Interest:", (2000 * 2 * 5) / 100)
```

Expected Output:

A revised Python program using constants such as RATE and TIME for clearer and maintainable calculations.

PROMPT:

Refactor the following Python code by replacing hardcoded numeric values with descriptive constants.

Define constants (such as RATE and TIME) at the beginning of the program and update the calculations to use them.

Ensure the output remains the same and keep the code clear and maintainable.

```
print("Simple Interest:", (1000 * 2 * 5) / 100)
```

```
print("Simple Interest:", (2000 * 2 * 5) / 100)
```

CODE:

```
# Define constants
PRINCIPAL_1 = 1000
PRINCIPAL_2 = 2000
RATE = 2
TIME = 5

# Calculate and print simple interest using constants
simple_interest_1 = (PRINCIPAL_1 * RATE * TIME) / 100
simple_interest_2 = (PRINCIPAL_2 * RATE * TIME) / 100
print("Simple Interest for Principal 1:", simple_interest_1)
print("Simple Interest for Principal 2:", simple_interest_2)
```

OUTPUT:

```
Simple Interest for Principal 1: 100.0
Simple Interest for Principal 2: 200.0
```

EXPLANATION:

This program calculates simple interest using constants instead of hardcoded values.

Constants like PRINCIPAL_1, PRINCIPAL_2, RATE, and TIME are defined to make the code clearer and easier to maintain. The simple interest is then calculated using the formula $(\text{Principal} \times \text{Rate} \times \text{Time}) / 100$ and printed.

Task Description -5 (Re factoring – Enhancing Variable Naming and Code Clarity)

- Task: Apply AI tools to improve code readability by renaming unclear variables and adding meaningful comments..

Instructions:

- Replace non-descriptive variable names with meaningful identifiers.
- Add inline comments where the logic may be unclear.
- Ensure program output remains the same.

Sample Code:

```
x = 50
```

```
y = 30
```

```
z = x + y
```



```
print(z)
```

Expected Output:

A Python program with clear variable names and explanatory comments that improve readability without altering functionality.

PROMPT:

Refactor the following Python code to improve readability.

Rename unclear variables with meaningful names and add comments explaining the logic.

Ensure the program output remains the same.

```
x = 50
```

```
y = 30
```

```
z = x + y
```

```
print(z)
```

CODE:

```
# Define the first number
first_number = 50
# Define the second number
second_number = 30
# Calculate the sum of the two numbers
total_sum = first_number + second_number
# Print the result
print(total_sum)
```

OUTPUT:

```
80
```

EXPLANATION:

This program adds two numbers using clear variable names and comments.

Variables `first_number` and `second_number` store the values 50 and 30, and `total_sum` stores their addition. Comments explain each step, and the program finally prints the sum (80).